ClawTasker CEO Console

Release candidate focused on a modernized CEO Console shell, mission planning for shared briefs and staffing/risk visibility, agent self-registration for the company chart, Pocket Office Quest v9 day/night office scenes, protected office-object bounds, and a thin operator conversation rail aligned to OpenClaw channels and transcripts.

Release	OpenClaw target	Runtime role
4.2.0-rc3	2026.3.13 / v2026.3.13-1	Visualization companion with restart-safe local recovery

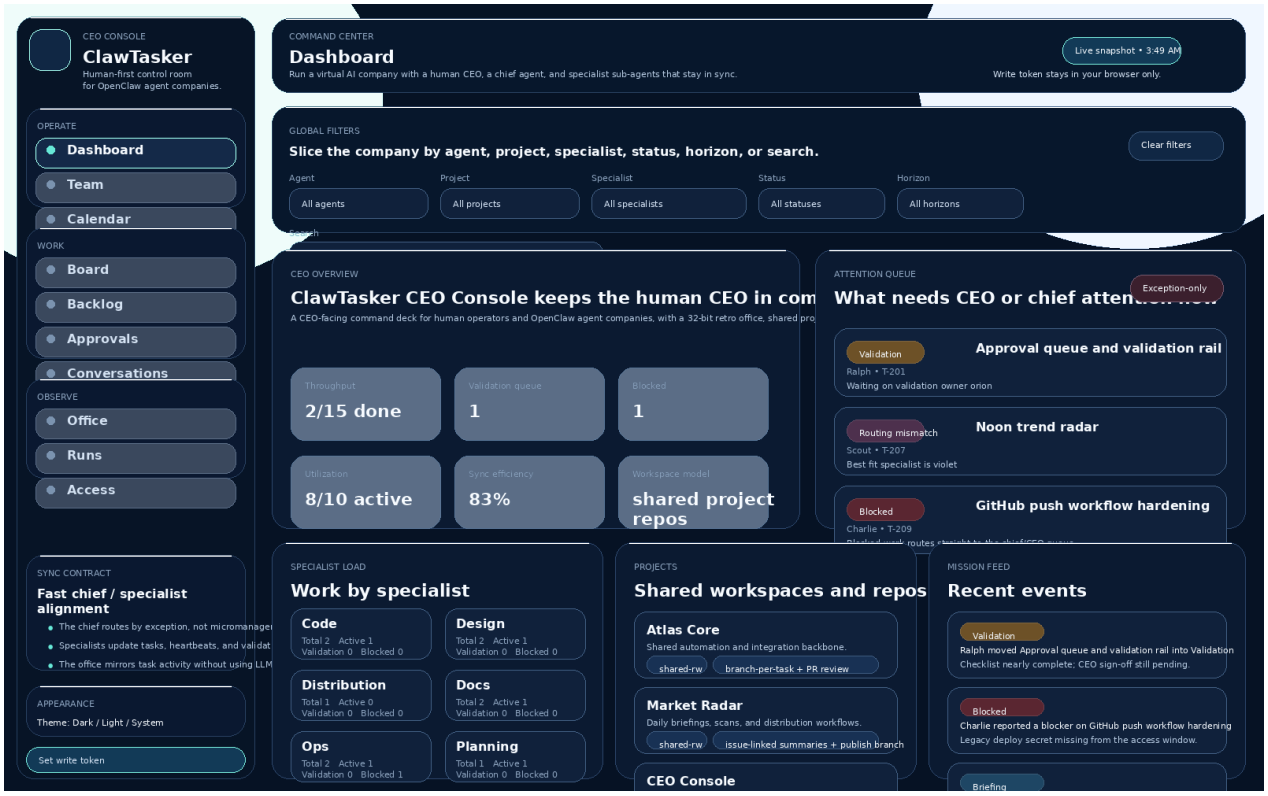

Figure 1. OpenClaw-style shell with grouped navigation, Appearance controls, system health, and CEO-facing workflow views.

1. Executive summary

- ClawTasker remains a standalone local-first product: one Python server plus a prebuilt static frontend bundle, with no runtime cloning or migrations.

- This release candidate focuses on a screenshot-aligned CEO Console shell, first-class mission planning, Pocket Office Quest v9 character integration, visible-face roster cards, agent self-registration for company-chart visibility, deterministic office-area placement, protected office-object bounds, day/night office scenes, and a thin operator conversation rail built on top of OpenClaw-native channels/transcripts.

- The platform boundary remains explicit: ClawTasker is a visualization and collaboration companion, while OpenClaw keeps routing, sessions, subagents, and workspaces.

- The office is deterministic and token-free: task state drives desk, sync-table, and review-rail placement without using an LLM for animation, and movement now respects protected furniture bounds.

- The release package includes updated requirements, traceability, regression tests, raw results, renders, a PDF guide, a mission-planning assessment, an agent API guide, a Mission Control prompt pack, and an OpenClaw companion pack with explicit conversation-source/channel metadata plus inspectable office layout rules.

2. Pocket Office Quest v9 integration

The supplied Pocket Office Quest v9 character pack is vendored directly into the release, while the shell now boots into the CEO Console palette by default. ClawTasker adapts the portraits for roster/profile views, builds office sprite strips from the supplied sheets, and generates compatible day/night office layouts so the runtime can expose office-scene switching even though the supplied archive does not include standalone office background bitmaps.

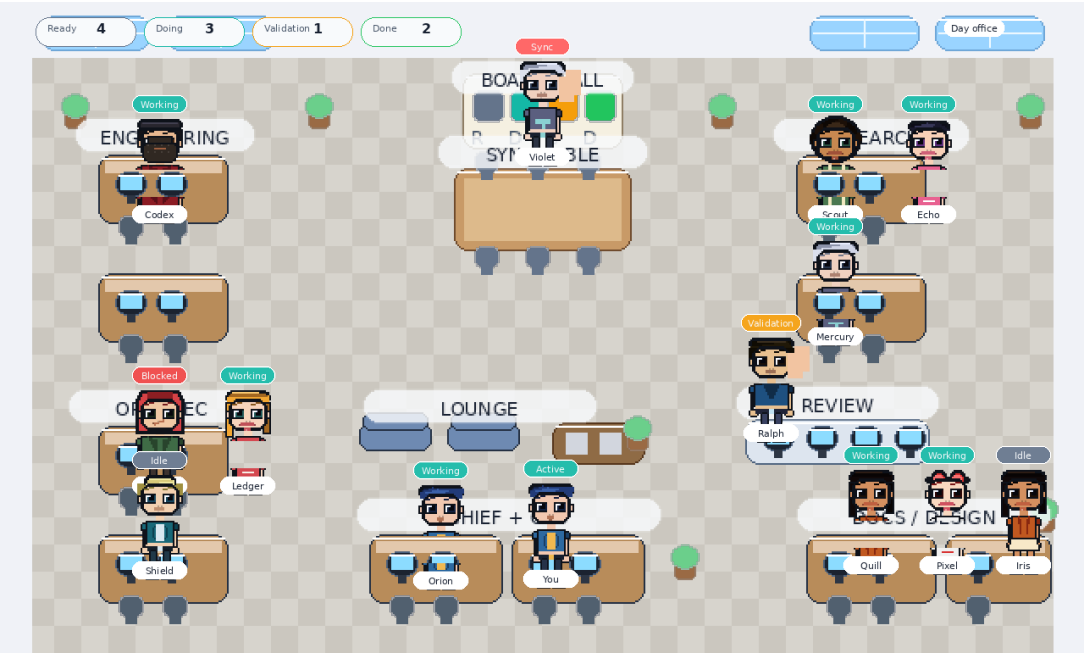

Figure 2. Day office scene with deterministic zone placement, visible status labels, and manager/specialist presence.

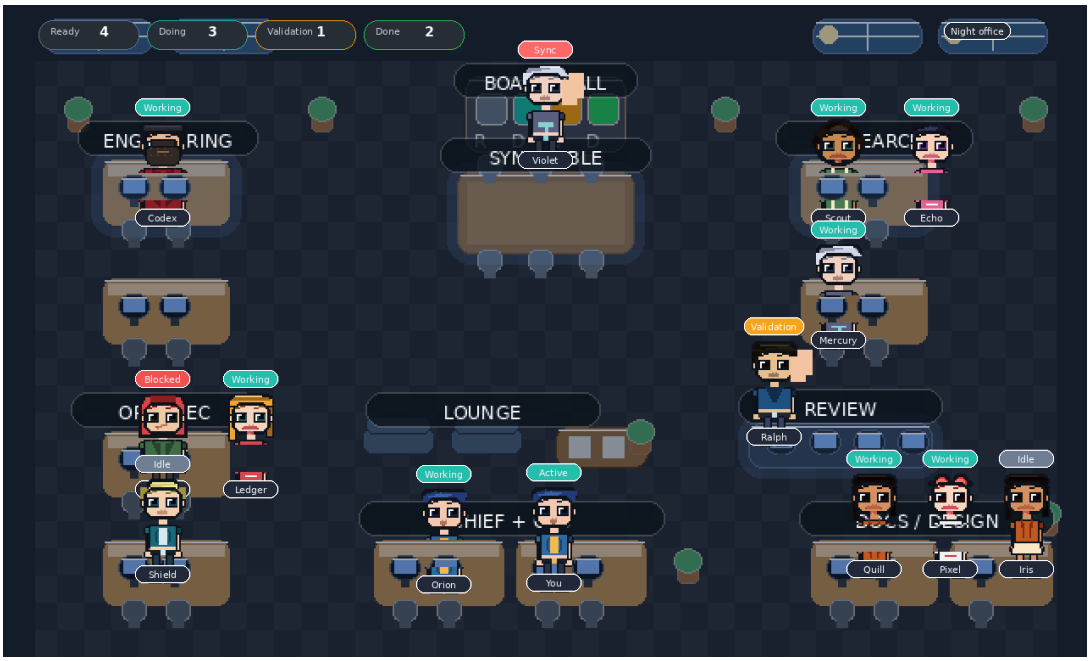

Figure 3. Night office scene using the same roster state and office-area layout with alternate lighting.

3. Office areas and movement rules

- The office is organized into explicit zones and protected object bounds: CEO strip, chief desk, engineering pod, research pod, ops pod, QA pod, studio pod, sync table, review rail, board wall, and lounge.

- Working agents stay in desk pods, coordination moves agents to the sync table, and validation moves agents to the review rail.

- Placement is collision-safe: visible agents receive unique seat anchors, protected furniture bounds are exposed in the snapshot, and the regression suite validates that office placements do not overlap or spawn inside office objects.

- The office engine respects layout intent rather than freeform wandering, keeping the view readable for a human operator.

Figure 4. Full office floor showing day/night layout compatibility assets and named office areas.

4. Visible-face roster and company structure

Portraits and office sprites come from the Pocket Office Quest v9 adaptation pipeline. The roster render emphasizes readable faces, larger hero poses, and directional rows, while the company chart supports the human CEO, the chief agent, multiple managers, and synced specialist roles such as HR, Purchasing, or Media Analysis.

Figure 5. Visible-face roster cards with readable portraits, hero poses, and directional sprite strips derived from the v9 pack.

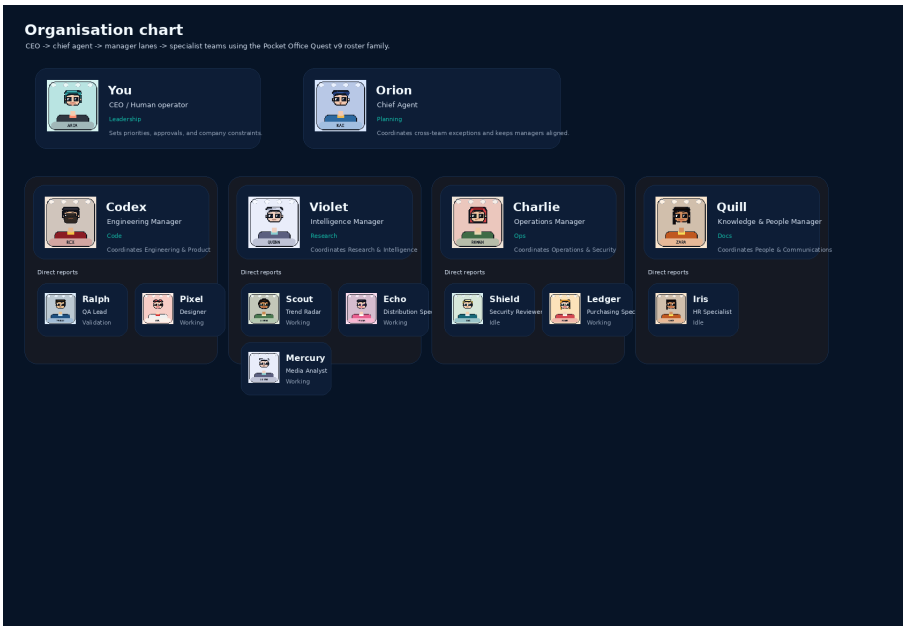

Figure 6. CEO -> chief agent -> manager lanes -> specialist teams using the integrated Pocket Office v9 avatar family.

5. OpenClaw alignment and operator boundary

- OpenClaw remains the source of truth for agent creation through agents.list-style infrastructure; ClawTasker receives roster sync envelopes and lightweight self-registration payloads over a dedicated local API.

- The OpenClaw bridge is local-first and restart-safe: if ClawTasker restarts, agents continue working and can republish status later.

- Task lifecycle guardrails remain in place: deterministic ordering, validator enforcement, owner validation, retry-safe publish dedupe, and mission-linked task tracking.

- The office view is intentionally bounded for readability and performance; the Team view remains authoritative for larger rosters.

Metric	Meaning
Open items	All tasks not yet in done
Mission coverage	Coverage of required specialists on the focus mission
Open risks	Current mission risks that are still open or unmitigated
Assignment drift	Active tasks whose owner is not currently aligned in the roster
Missing validators	Validation/done tasks lacking a validator
Roster sync	Last synced roster metadata published from OpenClaw infrastructure
Office overflow	Agents beyond the office visual capacity that remain visible through Team filters
Publish dedupe	Seconds during which an identical OpenClaw publish envelope is treated as a retry

6. Conversation rail and official channels

- Conversation source badges show whether a thread or message came from browser chat, Telegram, Discord, a webhook, or an internal OpenClaw session.

- The conversation surface is intentionally thin: directives and discussion are separated, and subagent-to-subagent threads default to summaries only unless a human chooses to reveal more detail.

- Where available, each thread exposes official-channel handoff links plus session key, run ID, transcript path, and transcript URL references so audit trails stay tied to OpenClaw rather than being duplicated here.

- This keeps Mission Control focused on human visibility and collaboration while OpenClaw remains the runtime communication engine for agent work.

7. Appearance, palette, and office scenes

The shell follows the OpenClaw palette direction and grouped shell proportions while refreshing the dashboard to match the supplied command-center reference more closely. The Appearance view includes mode, palette family, accent tuning, CEO profile controls, and a persistent office scene selector.

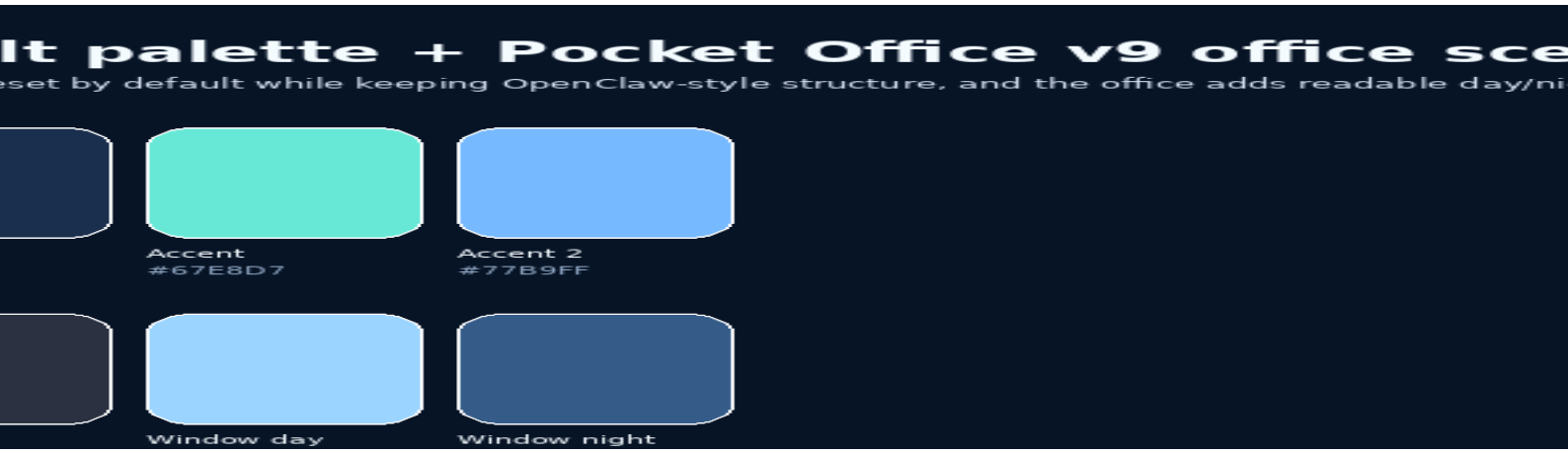

Figure 7. OpenClaw-style shell palette tokens plus office day/night scene switching.

8. Run and validate

Start the app with one command, then open the loopback URL in a browser.

```
export CLAWTASKER_API_TOKEN="replace-this-with-a-strong-local-token"
python3 server.py
```

Open:

```
http://127.0.0.1:3000
```

Validation commands:

```
python3 scripts/adapt_pocket_office_release.py
python3 scripts/build_static_ui.py
python3 docs/build_guide.py
python3 -m py_compile server.py
python3 -m unittest discover -s tests -v
node --test ui/tests/*.test.mjs
bash scripts/smoke_test.sh
```

9. Publication package

- The repo includes README, MIT license, changelog, security policy, contributing guide, support notes, and GitHub issue/PR templates.

- The release ZIP includes the OpenClaw companion pack, the vendored Pocket Office Quest v9 character library, the Mission Control prompt pack, the agent API guide, requirements, traceability, renders, the regression report, and raw test results.

- This release candidate is intended for validation of the v9 office/roster integration while staying compatible with the latest stable OpenClaw line.